Selective Replay Under Incomplete Dependency Information

An exploratory benchmark of correctness and audit cost

Matthew Looney - Magnexis

ORCID: https://orcid.org/0009-0007-0025-8000

October 21, 2026 - Draft v0.2 - Not peer reviewed

Abstract

Reusing intermediate results can reduce the work required to rerun a computational workflow, but its correctness depends on identifying the consequences of changed inputs. We investigate whether sparse, fresh-execution audits can recover enough correctness lost through incomplete dependency information to retain a useful work reduction. Two dependency-free Python studies compare full replay, transitive invalidation, and two sampled-audit policies: a 480-scenario in-memory DAG study and a 96-scenario controlled file-processing study that performs real reads and writes. With complete dependencies, transitive invalidation matches full replay in every observed scenario. With 5% independent edge omission, the in-memory 10% audit reduces incorrect runs from 74/160 to 46/160 at 70.7% mean task work; the file-processing 10% audit reduces them from 13/32 to 7/32 at 79.3% mean task work. At 20% omission, the file-processing audit leaves 7/32 incorrect runs and exceeds full replay task work. The evidence therefore rejects any assumption that sparse audit sampling alone makes incomplete dependency information safe. It supports a narrower conditional result: selective replay is correct only when every relevant input is represented, and audit sampling is a diagnostic mechanism with an explicit cost/reliability tradeoff. This is an exploratory systems study, not a proof of general workflow correctness or evidence about deployed AI agents.

1. Motivation and research question

Developer tools, build pipelines, and local automation frequently repeat work after a small edit. Reusing previous task outputs is attractive on machines with limited compute. However, an implementation can save work by accidentally preserving stale output. A useful replay policy must therefore be judged jointly on correctness and cost. Measuring work savings alone rewards unsound policies.

Our question is: when a workflow's declared dependency structure omits some actual reads, how much correctness can a sampled fresh-execution audit recover, and what work does that recovery consume? A future application could be a local automation runtime that records dependencies and supports selective reruns. The present implementation is a deterministic simulator; it neither executes arbitrary projects nor integrates an LLM.

The unresolved hypothesis is that there exist practically useful workload classes for which sparse auditing reduces the incorrect-run rate by at least half while using no more than 75% of full-replay task work. These thresholds describe an experimental target, not an accepted reliability standard. Our pilot does not meet both thresholds in either omission condition. Further work should identify conditions of

success or reject the hypothesis rather than tune a demonstration until it appears successful.

2. Relationship to existing work

Dependency tracking and incremental computation are established research topics. Mokhov, Mitchell, and Peyton Jones [1] develop an executable framework for comparing build systems. Coetzee, Bhaskar, and Necula [2] discuss how manual dependency specifications can produce invalid outputs and outline a resource-oriented model for reliable builds. Lyu and colleagues [3] investigate dependency-error detection for incremental C/C++ builds using EChecker. Acar, Blume, and Donham [4] give a semantics and correctness treatment for self-adjusting computation.

We do not claim to invent incremental computation, dependency closure, sampled validation, or correctness under complete dependency tracking. This pilot contributes a small reproducible fault-injection harness and explicit accounting for the work of fresh audits and fallback. A comprehensive literature review and comparison against real dependency-tracking systems remain necessary before making a novelty claim. Unlike EChecker, our benchmark does not infer dependencies from program text or evaluate real build histories. Unlike a formal semantics for self-adjusting computation, it does not establish correctness for dynamic or effectful programs.

3. Model and restricted correctness argument

Let G=(V,E) be a finite directed acyclic graph with a fixed topological order. Every node is a deterministic task. An edge u -> v means that v reads u's output. A task output is a SHA-256 digest of its node identifier, local input salt, and ordered parent outputs. Dependencies and node functions remain fixed between runs; one local input salt changes. The declared graph D is a subgraph of G and may omit edges.

The full-replay reference evaluates every node in topological order. The transitive policy reevaluates the changed node and every node reachable from it in D. Other nodes reuse their previous outputs. Correctness means equality of the entire vector of task outputs with full replay, rather than equality only at designated sink nodes. An incorrect run contains at least one unequal output.

Proposition (restricted reuse correctness). If D=G, all task functions are deterministic and unchanged, all local input changes are included in a set C, and there are no unrepresented external inputs, reevaluating C and its descendants yields the same output vector as full replay.

Argument. Proceed through the topological order. A node outside the affected set has unchanged local input and no affected ancestor. Its parents and therefore its output are unchanged. A node inside the affected set is reevaluated after its parents, whose values agree with full replay by induction. Applying the same deterministic function yields the same value. This argument does not require hashing; it follows from the stated data dependencies. It is a standard dependency argument included to define the benchmark's boundary, not a claimed new theorem.

If D omits an edge, the argument can fail. In a three-node chain a -> b -> c, changing a while failing to invalidate c can retain an old result even if b is rerun. External state, clocks, randomness, network responses, changing task code, and effects must be represented or controlled before the proposition applies to a real system. A sampled audit provides no deterministic guarantee of detecting all violations.

4. Experiment design

The pilot uses 40 seeds (0 through 39), two sizes (64 and 256 tasks), two maximum fan-in values (1 and 3), and three edge-omission probabilities (0%, 5%, and 20%). This gives 480 scenarios, each evaluated under five policies. For node i, actual parents are uniformly sampled without replacement from preceding node identifiers, with count min(i, fan-in). This is an ordered random DAG model, not a uniform sample of all DAGs. Early nodes consequently have fewer parents. The changed node is selected uniformly from the first quarter of nodes, deliberately increasing potential downstream impact.

Each scenario first computes an all-zero-salt baseline, changes one salt to one, and computes a fresh reference. A separate seeded random stream independently drops edges from the declared graph. Reusing the same streams across omission probabilities couples the scenarios and produces nested declared edge sets. Seeds are also reused across sizes and fan-in settings. Accordingly, the 160 scenarios per omission condition must not be treated as 160 independent real-world observations.

The five policies are: full replay; direct-only invalidation (changed nodes and immediate declared children); transitive declared invalidation; and transitive invalidation followed by either a 2% or 10% cached-node audit. Direct-only invalidation is an intentionally weak diagnostic control, not a serious replacement for established incremental systems.

An audit uniformly samples ceil(q times cached-node count) cached nodes without replacement. It freshly evaluates each sampled node and its actual ancestors, memoizing this fresh context independently of the candidate execution. If a mismatch is found, it stops sampling and finishes full replay in that fresh context. On no mismatch, it returns the candidate output vector. The simulator exposes actual parent reads during fresh execution; a practical implementation would need equivalent instrumentation or isolated task execution. It does not let the selective scheduler see actual edges beforehand.

Work is the number of task evaluations divided by task count in full replay. Audit work includes all fresh ancestor evaluations. If a task was run during both selective execution and the fresh audit, both executions count. A fallback reuses work already completed in the fresh audit context, but not the candidate context. Values above 1.0 are possible. Graph traversal, storage, instrumentation, comparison, and scheduling costs are excluded; these are task-count results, not measured latency or energy savings. Reference construction is evaluation infrastructure and is excluded equally for all policies.

5. Pilot results

The table pools the four size/fan-in combinations equally, with 160 scenarios per omission condition. Each work ratio is calculated within a scenario before averaging. An incorrect-run percentage describes this finite benchmark grid, not an estimated production failure probability.

Missing edges	Policy	Incorrect runs / 160	Mean work / full
0%	Full	0	1.000
0%	Direct only	137	0.056
0%	Transitive	0	0.363

Missing edges	Policy	Incorrect runs / 160	Mean work / full
0%	Audit 2%	0	0.470
0%	Audit 10%	0	0.588
5%	Full	0	1.000
5%	Direct only	140	0.054
5%	Transitive	74	0.342
5%	Audit 2%	58	0.540
5%	Audit 10%	46	0.707
20%	Full	0	1.000
20%	Direct only	143	0.046
20%	Transitive	125	0.263
20%	Audit 2%	84	0.611
20%	Audit 10%	47	0.880

With complete dependencies, transitive invalidation requires 63.7% fewer task evaluations on average than full replay. The zero mismatches are consistent with the restricted argument; the theoretical conditions, not the finite test count, justify that argument. Direct-only invalidation often fails despite its small task count, illustrating why cost cannot be interpreted alone.

At 5% omission, the 10% audit removes 28 of the 74 incorrect runs remaining after transitive invalidation, a 37.8% reduction. This falls short of the proposed 50% target, although the mean work ratio remains below 0.75. At 20% omission, it removes 78 of 125 incorrect runs, a 62.4% reduction, but exceeds the work target. Even there, 47 of 160 runs remain incorrect. Thus neither nonzero omission condition satisfies both targets. No hypothesis tests, confidence intervals, or production reliability claims are inferred from these exploratory pooled counts.

6. Controlled file-processing experiment

The simulation determines task outputs in memory. To determine whether the same behavior persists when actual filesystem activity is included, we added a controlled on-disk experiment. It creates only temporary synthetic inputs and outputs; it does not inspect or execute repository code, user files, shell commands, or network services. Each task reads a dedicated UTF-8 input file and the output files of its actual parents, then writes a SHA-256 output file. The payload is intentionally simple, deterministic, and side-effect-free apart from files in the temporary experiment directory. A fresh full replay after changing one input is the reference vector.

The declared graph supplied to the scheduler may omit actual parent reads, as in the first study. For a selected replay, unselected task outputs are copied from the baseline output directory; selected tasks recompute from actual input and parent-output files. The audit samples cached nodes, recursively

recomputes their actual ancestors in a separate fresh directory, compares sampled fresh outputs against candidate outputs, and falls back to a full fresh replay when it observes a mismatch. Work counts task executions; copies, traversal, comparisons, directory creation, and cleanup are excluded from that count. Elapsed time is retained as a secondary descriptive measure that includes these overheads.

The executed protocol contains 96 scenarios: eight seeds (0-7), two sizes (32 and 128 tasks), two maximum fan-in values (1 and 3), and three independent declared-edge omission probabilities (0%, 5%, 20%). Four policies are evaluated for each scenario: full replay, declared transitive replay, and audits of 2% and 10% of cached nodes. This design revision was made after an initial 20-seed, 64/256-task run proved unnecessarily slow in the Windows sandbox and was stopped before results were written. The reduced design, source, raw rows, and progress behavior are recorded in the repository. It remains an exploratory convenience grid, not a preregistered sample or a benchmark of arbitrary build tools.

The file experiment answers a narrower operational question than the first study: do omitted dependencies still lead to incorrect reused output when a workflow is actually reading and writing files, and do audit costs remain visible when I/O is included? It does not model process isolation, external tools, file metadata, timestamps, cache eviction, concurrent writers, symbolic links, or non-deterministic programs. It should be read as a controlled executable example rather than a substitute for a real-project corpus.

7. Controlled file-processing results

Table 2 reports all 32 scenarios per omission condition, pooled across sizes and fan-in. Timing figures are median wall-clock seconds on the run host and are descriptive only: they are below a second, sensitive to the filesystem and Python runtime, and not appropriate for cross-machine speed claims. A task-work fraction above 1 occurs because the candidate run and fresh audit can execute overlapping tasks separately; that is deliberate accounting rather than an error.

Missing edges	Policy	Incorrect runs / 32	Mean task work / full	Median seconds
0%	Full	0	1.000	0.0709
0%	Transitive	0	0.440	0.0656
0%	Audit 2%	0	0.545	0.1325
0%	Audit 10%	0	0.632	0.0961
5%	Full	0	1.000	0.0690
5%	Transitive	13	0.370	0.0972
5%	Audit 2%	8	0.655	0.1460
5%	Audit 10%	7	0.793	0.1129
20%	Full	0	1.000	0.0700
20%	Transitive	22	0.299	0.0944

Missing edges	Policy	Incorrect runs / 32	Mean task work / full	Median seconds
20%	Audit 2%	14	0.695	0.1477
20%	Audit 10%	7	1.056	0.1234

The on-disk experiment reproduces the qualitative result of the in-memory study. Correct declared transitive closure preserves the full result while avoiding many task executions. Incomplete declarations produce stale outputs in 13/32 scenarios at 5% omission and 22/32 at 20% omission. The 10% audit detects enough cases to reduce those counts to 7/32 in both conditions, but it does not eliminate them. At the higher omission rate it costs more task work than simply replaying everything. The 5% condition reduces errors by 46.2% (6 of 13), narrowly below the exploratory 50% target and above the 75% task-work target. The 20% condition reduces errors by 68.2% (15 of 22), but violates the task-work target. The result is not a positive demonstration of the joint hypothesis.

The wall-clock medians also show why task count and latency must be reported separately. At complete dependency information, transitive replay executes fewer tasks but has a median 0.0656 seconds compared with 0.0709 seconds for full replay in this small workload; this gap is too small and environment-dependent to support a speed claim. The 2% audit has a higher median time in every omission condition. The 10% audit can have a lower median than the 2% audit because it detects a mismatch earlier in some scenarios and switches to a sequential fresh replay with different I/O behavior. That non-monotonicity is a property of this small implementation, not evidence that more auditing is faster.

8. What the combined experiment establishes

The larger experiment has three intended claims with different evidentiary status.

First, there is a conditional correctness claim. Given a finite acyclic workflow, deterministic tasks, fixed code, a complete dependency/input record, and a correct transitive closure, selective replay produces the same outputs as full replay. Section 3 supplies a direct induction argument for this restricted case. The two experiments are demonstrations consistent with that argument; they are not the proof.

Second, there is an empirical falsification result for a tempting engineering assumption: "a modest random audit makes a partially observed dependency graph safe enough to reuse." Both executed studies supply counterexamples. At nonzero edge omission, sampled audits leave stale output in the measured runs. The file study shows that this is not solely an artifact of returning values from an in-memory graph function; it occurs in the controlled file workflow as well. This does not quantify the probability of failure in all real workflows, but it is enough to rule out a blanket safety claim for this policy.

Third, there is a product hypothesis, not a conclusion: a local-first workflow runtime may become useful if it records all relevant reads, isolates side effects, handles dynamic inputs, and treats audits as escalation triggers rather than correctness certificates. The experiments do not prove that such a runtime is performant, secure, commercially useful, or general enough for arbitrary developer tools. They identify the burden of proof for the next engineering phase: show complete input tracking on real

histories, compare against established incremental systems, measure end-to-end overhead, and demonstrate that unsupported effects fail closed rather than silently reuse output.

This distinction matters for the proposed larger experiment. The work should not be presented as a theory that "proves caching works," nor as evidence that an autonomous developer agent can safely rerun arbitrary tasks. Its useful contribution is a disciplined negative boundary: incomplete dependency information plus sparse random checking is not a substitute for complete provenance. The likely path forward is therefore instrumentation and explicit uncertainty, not a more optimistic cache policy.

9. Threats to validity

Hash-based tasks are highly change-sensitive: a parent change almost always changes downstream output. Real programs may normalize, ignore, or cancel inputs. This makes the simulator useful as a propagation stress test but a poor direct model of semantic equivalence. Cryptographic collision probability is not analyzed empirically. All tasks have unit cost, while real costs can differ by orders of magnitude. No cycles, concurrency, retries, failures, changing topology, incremental file reads, persistent side effects, or nondeterministic tasks are modeled.

Uniform independent omission does not capture structured failures such as an unrecorded environment variable used across many tasks. The restriction to early changed nodes biases the effect distribution. Complete access to actual dependencies during fresh audit is a substantial implementation assumption. Recursively rerunning ancestors may be infeasible for external services or unsafe for effectful tasks. Auditing must not repeat irreversible actions. The benchmark validates neither sandboxing nor security against malicious workloads.

Pooling obscures size and topology differences; raw rows preserve those strata. Shared seeds create paired observations. No external replication has occurred. The implementation and manuscript were developed together with AI assistance, and the pilot was examined before the prospective protocol below was written. This study is not preregistered. The narrow related-work review does not establish originality.

10. Prospective validation and engineering work

Freeze the current pilot and protocol before a separate validation run. Use unseen seeds 40 through 239; sizes 128 and 512; fan-in 1, 3, and 6; and omission probabilities 1%, 5%, and 20%. Include both independent omission and a documented clustered-omission mechanism. Select changed nodes from early, middle, and late graph regions in separate strata. Add heterogeneous task costs and workloads with semantically unchanged outputs.

The primary endpoint should test the 10% audit at 5% independent omission: at least a 50% reduction in incorrect runs relative to transitive invalidation and mean total task-work ratio at most 0.75. Freeze stratum weights before running. Treat seed as the resampling cluster for paired uncertainty intervals; report the correctness difference and work difference as well as the joint target. Clustered omissions, alternative audit budgets, mutation positions, and other rates are secondary analyses. Report failed targets without redefining them after results are seen.

Before practical claims, implement read tracking for a small local, side-effect-free workflow runner, replay real edit histories, and compare against full execution and an established incremental baseline.

Measure wall time and metadata overhead separately. Evaluate audit strategies against random sampling without assuming a new strategy is superior. Conduct a broader literature review and solicit independent reproduction. A publishable contribution may ultimately be a negative result about audit economics rather than a successful general-purpose replay method.

11. Reproducibility and availability

The companion repository is https://github.com/theworker02/replay-integrity-lab. It is initially private during author review. A public artifact snapshot and persistent identifier are prerequisites for an independently reproducible public preprint. No DOI has been issued for this draft. The repository includes `experiment.py` and `file_benchmark.py`, their raw CSV outputs, summary JSON files, checksums for the simulation inputs/results, test suites, and a reproducible manuscript source. The source is intentionally dependency-free; Python 3.10 or later is sufficient.

Run `python -m unittest discover -v`, `python experiment.py`, and `python file_benchmark.py` from the repository root. The first benchmark regenerates `results/trials.csv`, `results/summary.json`, and `results/manifest.json`; the second regenerates `results/file_trials.csv` and `results/file_summary.json`. Ten unit/regression tests cover transitive propagation, audit fallback, empty audits, repeatability, full-reference behavior, complete dependency tracking in the on-disk runner, and actual output changes after filesystem input mutation. Tests do not substitute for independent implementation or real-workload evaluation.

12. Conclusion

Selective replay is correct within a restricted deterministic model when all dependencies and input changes are represented. Incomplete dependencies break that guarantee. In this pilot, sparse fresh-execution audits reduce incorrect runs but leave substantial residual error and can approach full-replay work. The proposed cost-and-correctness hypothesis remains unresolved beyond this model and fails its joint target in the two nonzero-omission pilot conditions. The immediate research task is to validate or reject useful workload-specific regimes, then test an instrumented implementation on real histories.

Appendix A. Worked trace of an omitted dependency

This trace makes the failure mechanism concrete. Consider four deterministic tasks in topological order. Task A reads source file `theme.txt` and writes a normalized color token. Task B reads A and generates a stylesheet fragment. Task C reads A and creates a text report containing the token. Task D reads both B and C to assemble a package manifest. The actual graph is A -> B, A -> C, B -> D, and C -> D. Suppose the declared graph accidentally lacks the edge A -> C because C reads the source-derived token through an uninstrumented helper. The declared graph still includes A -> B, B -> D, and C -> D.

Task	Actual reads	Declared parents	Baseline output	Output after `theme.txt` changes
A	`theme.txt`	none	`blue`	`amber`

Task	Actual reads	Declared parents	Baseline output	Output after `theme.txt` changes
B	A	A	CSS using `blue`	CSS using `amber`
C	A	none	report says `blue`	report says `amber`
D	B, C	B, C	manifest for blue assets	manifest for amber assets

When `theme.txt` changes, declared transitive closure starting at A chooses A, B, and D. C is excluded because the declaration says it has no relation to A. The selective run recomputes A and B, reuses C's old report, then recomputes D from the new stylesheet fragment and stale report. D's result differs from fresh full replay. Crucially, the manifest may still appear internally well-formed: it has a valid hash and a complete list of inputs according to the declared graph. Validity of a cache key does not prove completeness of the dependency record that fed it.

The direct-only policy is worse: it might run A and B but leave D cached too. The transitive policy detects the declared downstream route through B, which avoids one stale branch but cannot recover the missing route through C. This is why the baseline comparison in the experiments checks every task output against full replay rather than checking only whether the scheduler touched a plausible number of tasks. A correct-looking build log does not establish that every semantic consequence of a change has been captured.

Now apply the audit policy. Its candidate result contains a new A, new B, cached C, and new D. If the audit samples C, it recomputes C in a fresh context. Fresh C reads the new output of A and produces the amber report, which differs from cached C's blue report. The mismatch causes a full fresh replay, and the final vector matches reference. If the audit samples only B, its fresh value agrees with candidate B and the audit returns the candidate; C and D remain stale. If it samples D, it can detect the problem only when the fresh execution of D recursively uses fresh B and fresh C. This recursion is why the benchmark's audit constructs actual ancestors in a separate context. An implementation that validates a selected output against mixed fresh and cached parents can hide precisely the discrepancy it is meant to find.

The probability of detecting this fault is not simply the audit fraction. It depends on which cached tasks are sampled, whether stale effects reach sampled outputs, whether a sampled output is semantically sensitive to its stale input, and whether the fresh evaluation is safe and available. In the small graph, one of the two cached tasks may be C; in a larger graph, a missing edge may affect only a small region, or a shared external input may affect many regions. Uniform random sampling consequently has no universal error bound without assumptions about fault distribution and observability. The experiments deliberately do not manufacture such a bound.

For a practical local workflow system, this trace suggests four engineering requirements. First, record reads at the boundary where values enter a task, including files, environment values, tool versions, configuration, and declared network snapshots. Second, distinguish pure re-executable tasks from

effects such as publication, deployment, or mutation of a database; an audit must not repeat an irreversible effect merely to gain confidence. Third, give users an explicit state for "dependency evidence incomplete" and escalate to full replay or review rather than silently returning a cache hit. Fourth, preserve replay receipts: which inputs were observed, which outputs were reused, what audit sample was drawn, and why a task was allowed to reuse a prior value.

The worked trace does not introduce a new algorithm. Its purpose is to connect the small experimental outcome to the larger proposed system. The key property to build is not a cleverer random sampler. It is a trustworthy observation boundary that can justify selective reuse in the first place. Audits remain useful as anomaly detection and defense in depth, but they should not be advertised as a proof that missing provenance is harmless.

Declarations

Author: Matthew Looney, Magnexis. ORCID: 0009-0007-0025-8000. AI assistance: OpenAI Codex assisted with study design, implementation, execution, literature discovery, and drafting. Author review of references, claims, licenses, and the final manuscript is pending. No claim of independent human verification is made. No human-subject data were collected. Commercial interest: the author develops software through Magnexis, which may benefit from workflow research. No external funding is reported in this draft; the author should confirm that statement before submission. This manuscript is a working preprint draft, not an accepted or peer-reviewed publication. The v0.2 revision adds a controlled filesystem experiment conducted after the v0.1 draft. It does not alter the status of the work as exploratory.

References

[1] Andrey Mokhov, Neil Mitchell, and Simon Peyton Jones. 2018. Build Systems a la Carte. Proceedings of the ACM on Programming Languages, ICFP. https://www.microsoft.com/en-us/research/publication/build-systems-la-carte/

[2] Derrick Coetzee, Anand Bhaskar, and George Necula. 2012. A model and framework for reliable build systems. arXiv:1203.2704. https://arxiv.org/abs/1203.2704

[3] Jun Lyu, Shanshan Li, He Zhang, Yang Zhang, Guoping Rong, and Manuel Rigger. 2024. Detecting Build Dependency Errors in Incremental Builds. arXiv:2404.13295. https://arxiv.org/abs/2404.13295

[4] Umut A. Acar, Matthias Blume, and Jacob Donham. 2011. A Consistent Semantics of Self-Adjusting Computation. arXiv:1106.0478. https://arxiv.org/abs/1106.0478